Reinforcement learning for African-language code-switching and idiom generation: a learned reward alone is not enough

*Working draft, 2026-10-31. Numbers marked **[pending]** are from runs that have not finished. Everything else is taken from logged outputs in* `reports/data/` *.*

Abstract

We fine-tune a 301M-parameter decoder (the SFT model) to write human-style code-switched text and idiom-rich text in 37 African languages, and we use reinforcement learning (GRPO) to improve it. The reward is a learned, reference-free evaluator (a fine-tuned mmBERT-base with 27 rating heads) combined with deterministic rule penalties. Our main finding is a negative one about the reward: when the evaluator alone supplied the reward (plus a small set of rules), the proxy reward rose from 0.37 to 0.61 while an independent reference metric fell by about 7 chrF++ on the two code-switching tasks, and English leaked into the output. The failure is invisible to the evaluator, which we show is also a weak judge of translation quality against human ratings (Spearman 0.25 vs 0.49 for SSA-COMET-QE) and blind to hallucinated additions. Adding the missing rules (language-ID guards, function-word and stop-word checks, diacritic, name, number, length, repetition and device-use penalties), a reference-free translation metric, fresh prompts that never occur in the SFT data, and a held-out chrF monitor limits the degradation. On the full SFT test set (6,230 items, 34 languages, greedy decoding) the final policy changes chrF++ by +0.8 to +1.3 on the three idiom tasks and by -0.7 and -1.0 on the two code-switching tasks; format validity rises from about 95% to 99.8%; and the mean reward rises from 0.14-0.53 to 0.28-0.74. Only about a quarter of that reward gain comes from the learned evaluator; about three quarters comes from the rules. An independent comparison with a single LLM judge (Gemini 3.8 Flash, a standard system-blind rubric, 1,000 items) finds no significant overall change from the SFT model to the RL policy (-0.015 on a 1-5 scale, 95% CI [-0.055, +0.025]), a significant decline on one code-switching task (-0.24), and a gap of 1.2 points to the text of the data-generation model: the reward gains do not translate into judged quality. We also report a negative interpretability result: the six group tokens added to the evaluator did not specialise.

1. Introduction

Code-switching, mixing a base African language with a second language or with idiom, is the default register of many speakers and is almost absent from public training data. We study how far RL with a learned reward can push a small model in this setting. The question has a safety side: reward-model overoptimisation (Gao et al., 2023) is established for strong English reward models, but low-resource languages are exactly where reward models are weak. We report one controlled-in-spirit case study and the fixes that made RL safe to run.

Contributions: (1) a reference-free evaluator for five code-switching and idiom tasks plus translation, with a full analysis of what it can and cannot judge; (2) a documented reward-hacking episode with an independent metric that exposed it; (3) a rule-anchored reward and monitoring protocol that prevented it; (4) a negative result on per-group special tokens.

2. Tasks and data

Six tasks over 37 languages: **eng_m** (English to language, translation replay), **eng_hm** and **m_hm** (English or language text to human-style code-switched text with a declared switch language), **eng_ood**, **m_ood**, **hm_ood** (to idiom-rich text). Each code-switching or idiom output consists of a declared list of devices (the switched phrases or the idioms), followed by the final text, optionally preceded by an English draft. The switch languages per base language come from a ranked table supplied by the author (e.g. Fon: French, Yoruba).

Training data used (all disclosed here and in the model cards): the project's own generated code-switching and idiom corpus (SFT), the evaluator's 167k labelled records (LLM-judge labels, synthetic negatives and corruption chains), and for RL a **fresh prompt pool** (Section 6.1). Not used: SSA-MTE and AfriMTE (CC-BY-SA / CC-BY; benchmark only), NLLB-produced rows, TranslatePsy. The labels come from commercial LLM judges, so the weights are research-only.

3. The SFT model and its ablations

The generator is a Gemma-3-270M-based decoder first fine-tuned for English-to-African translation, with language and format tags added to the vocabulary, then fine-tuned on the code-switching and idiom data (832,421 training rows, 17,399 validation, 6,230 curated test rows; the text was generated with a 31B instruction model, filtered by rules and by two LLM judges on about 22% of rows, with judge-corrected copies added). All ablations below used a 20% stratified subsample (166,484 training rows, about 60M tokens), one seed per arm, 2 epochs, an identical recipe (AdamW 5e-5, cosine decay, 262,144-token optimizer steps, loss on completion tokens only), and chrF++ of the final text only (tags, draft and device block removed). The noise floor between two decodes of the same checkpoint is 1-1.6 chrF++, so single-run differences below about 1.5 are not reliable.

#	Question	Result	Evidence
1	Start from the translation-tuned model or from plain Gemma-3-270M?	translation-tuned wins: val loss 1.47 vs 1.91; chrF++ 35.1/38.6 vs 31.1; 20.7% looping outputs vs 0.7-9.3%	established (large gap)
2	Add 6 interleaved identity-initialised layers (301M) or 12 (335M)?	6 beats 12: 38.57 vs 37.70, 95% CI of the gap [+0.57, +1.17], better on all 5 tasks	established (same data, recipe, decoder, rows)
3	Does adding 6 layers beat the 18-layer baseline?	**no** at this data size: baseline 38.72 vs 38.57, CI of the gap [-0.16, +0.45]; an earlier partial comparison (+1.45 for the deeper model) disappeared on the full shared set	refuted by a clean retrain of both arms
4	Show the generator's English glosses next to each device?	hurts: -1.1 to -2.1 chrF++ on the three idiom tasks, more English leak	established
5	Give the model an optional English draft?	helps only English-to-idiom (+0.5 greedy / +0.8 sampled, inside the noise floor); hurts m_ood (-2.1)	mixed

Depth up-scaling arms (identity-initialised copies inserted after every third layer, new layers frozen out for the first 120 steps) and full results per task are in the results package. The deployed generator uses the 24-layer variant, trained on the full data; ablation 3 shows this was not demonstrated to be better than the 18-layer baseline at 20% data. Confounds: one seed per arm; the shared evaluation rows are the shortest prompts of each shard and language-skewed; two decoders (HF generate and vLLM) differ by about 3.5 chrF++ in absolute level, so only within-decoder comparisons are valid; the full-data effect of depth is unknown.

LLM judges. We validated candidate judges against human ratings before using them: on 300 SSA-MTE and AfriMTE segments, Gemini 3.8 Flash and 3.1 Pro reach Spearman 0.5-0.6 per language pair with human z-scores, comparable to SSA-COMET-MTL (reference-based), at about $0.001 and $0.005 per segment on the Batch API. Flash graded the training labels; the three judges agree with each other only at 0.6-0.7 on code-switching rows.

4. The evaluator

Architecture. mmBERT-base with six appended group tokens ([M] translation, [H] code-switching, [O] idiom, [D] devices, [X] extra, [P] overall) each feeding its own two-layer MLP head: 27 outputs on a 1-5 scale. Masked Huber loss; input contains task prefix, source, target, devices and (for half of ood training

examples) a draft. One run, optimizer state saved, early stopping with patience 5 (best validation Spearman 0.683 at step 22,500).

Agreement with LLM judges (Fig. 1). On the 4,175-row test set, the final evaluator beats the previous fine-tune against Flash on every task (+0.008 to +0.017, 95% CIs above zero) and against luna on every task (hm_ood within noise). On the 299 gold rows Spearman with Argon is 0.69 (hm) and 0.71 (ood). The three judges agree with each other at only about 0.6-0.7, which bounds the evaluator.

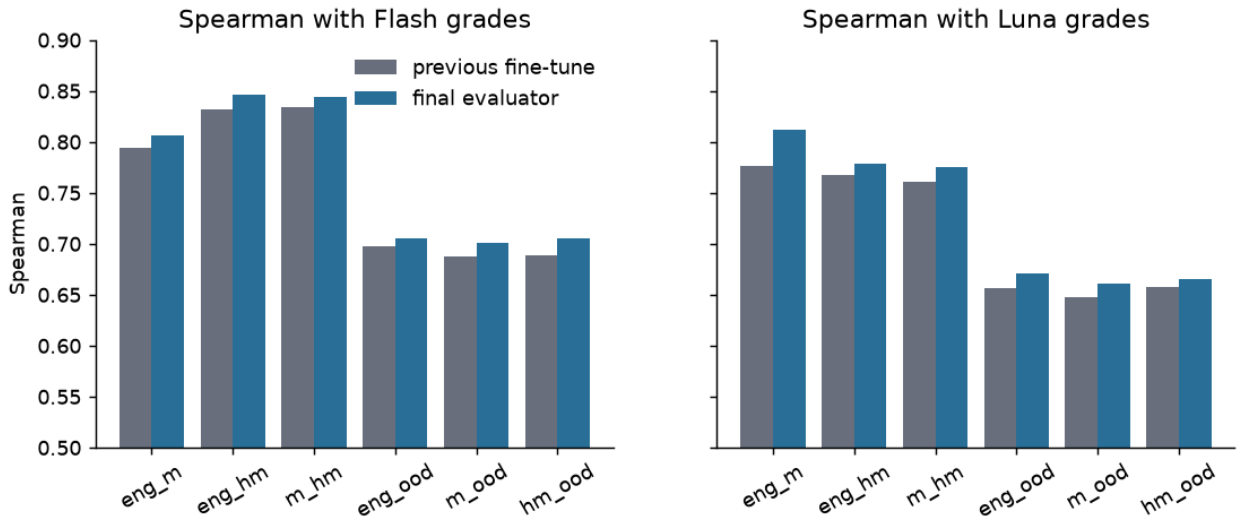


Fig. 1. Spearman agreement of the previous and final evaluator with Flash and luna grades, per task.

Structure checks. On 534 held-out bad-vs-contrast translation pairs the better one scores higher with probability 0.80; ordering accuracy on held-out corruption chains is 84% (translation), 89% (hm), 85% (ood).

Against humans (negative). On 20,204 human-rated eng-to-X segments (SSA-MTE test and AfriMTE) the evaluator's pooled Spearman is 0.25 (0.33 within language pair), below SSA-COMET-QE (0.49) and AfriCOMET-QE (0.35) (Fig. 3). It is a usable proxy for the LLM judges, not for humans.

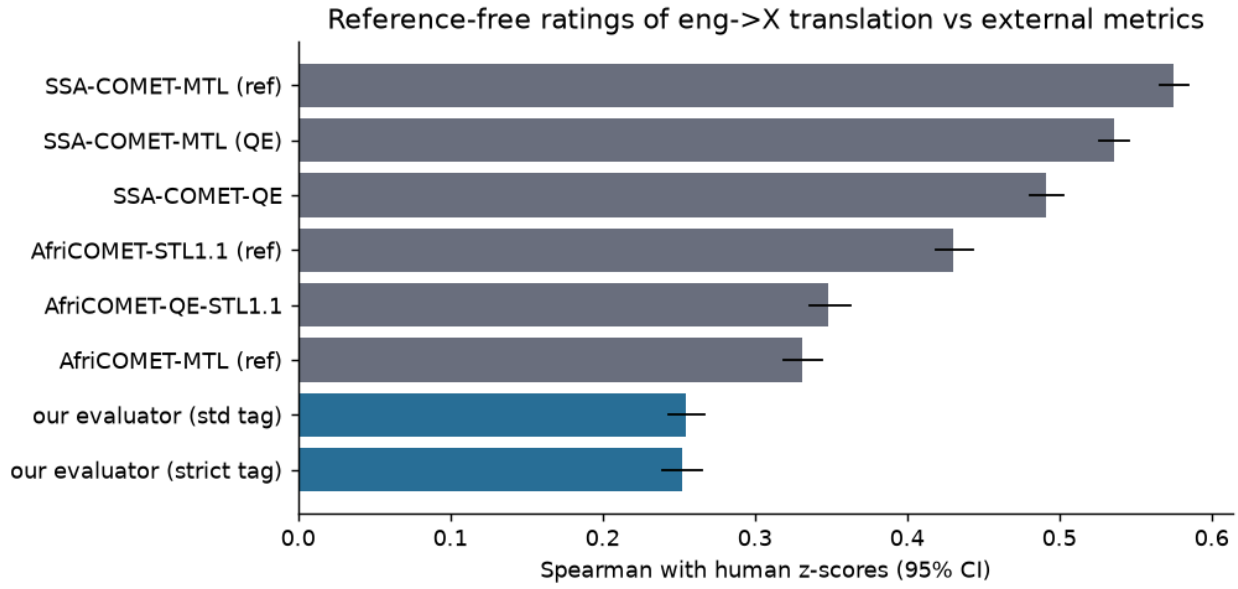


Fig. 3. Spearman with human ratings on 20,204 eng-to-X segments (95% CI): the evaluator against SSA-COMET and AfriCOMET variants.

Internals (Fig. 2). The six group tokens attend almost identically (shares differ by at most 0.04) and their final hidden states are near-identical (cosine about 1.0 between tokens), so the per-group tokens did not specialise;

the heads work through the shared pooled representation. Interventions: scrambling the target lowers every dimension by 1.7-2.4 points; shuffling the source lowers faithfulness-type outputs by 1.6-2.9; removing the devices block changes scores by less than 0.03 (the evaluator does not read devices); showing a draft raises ood scores by 0.07-0.14 (small, so the evaluator is robust to draft on or off).

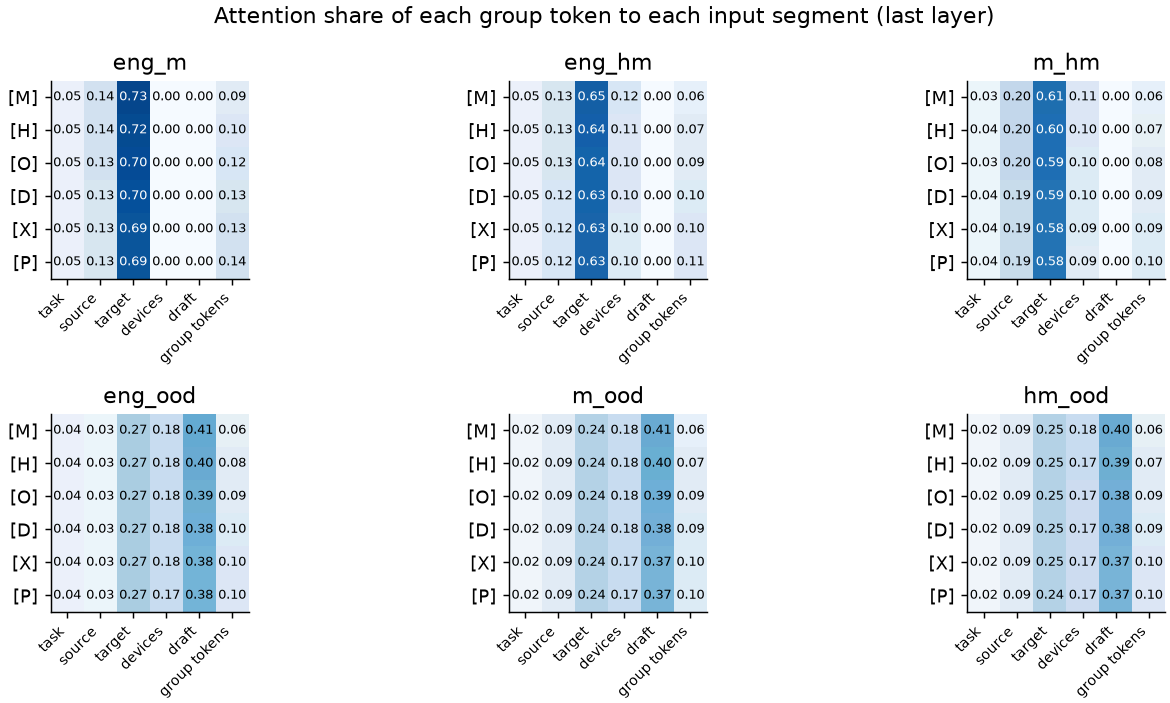


Fig. 2. Attention share of each group token to each input segment (last layer), per task.

5. RL method

GRPO with group size 8 (7 sampled at temperature 1 plus the greedy decode), 32 prompts per step (256 sequences, about 35k tokens), AdamW at 2e-6, token-level loss with a k3 KL estimator against the frozen SFT model, an adaptive KL coefficient (target 0.12, hard cut at 0.35), the greedy member's advantage computed against the sampled members' statistics and scaled by 0.3 (cap 3). vLLM samples with weights pushed each step.

Reward. `r = max(−1, (S−1)/4 − sum_k w_k p_k)` where S is the evaluator's equal-weight mean over the task's dimensions and `p_k in [0,1]` are rule penalties; a format failure is -1 and skips the evaluator. For eng_m in 14 languages covered by AfriCOMET, `(S−1)/4` is replaced by `0.3 (S−1)/4 + 0.7 q`, with `q` the reference-free AfriCOMET-QE score.

rule	weight	penalty
format gate	-1	missing end token, unclosed draft, no single `<dev>` block with 2+ devices, empty text
length	0.6	output/source character ratio outside a window (per task; per language for English-source tasks, 5th-95th percentile of the SFT references); draft removed first
loop	1.0	2-, 3-, 4-gram distinct shares below 0.80, 0.88, 0.90 (references are above 0.887, 0.96, 0.986)
number	0.8	changed numeral = 1; missing = 0.6 (0.25 if replaced by an English or language number word, 100 mined)
name	0.4	names of the English source absent from the output
english	0.8	English function-word share above 0.20 (ood, eng_m)
base_leak	0.8	switch-language function words above 12% outside the devices (hm)
base_presence	0.4	base-language function-word share below 0.6 x the human 5th percentile
switch	1.0	devices make up 18-45% of the words (hm), linear outside; mostly stop words penalised
device_use	0.6	listed devices that do not occur in the text
diacritic	0.6	density below the human 10th percentile, or marks outside the language's human inventory (pooled prior for languages without data)
lid	1.0	GlotLID sentence labels must be the base or (hm) switch language, weighted by classifier confidence (0 below 0.3, full at 0.8)
draft	0.3	draft size above 1.5x the source, loops in the draft

The greedy member's loop, length, lid and base_leak penalties are multiplied by 1.5. A language-ID estimate of the switch share is logged but not rewarded: GlotLID labels the declared devices as the switch language for only 33% of the human references.

Calibration. Each rule was run on 6,230 test references and on the SFT model's greedy outputs; rule rates are close for both (so references are not flagged) and far higher for the degraded run-2 outputs (base_leak > 0.15 on 32-36% of hm outputs vs 7-8%).

6. Experiments

6.1 Prompts and monitors

The first pool took its monolingual and code-switched sources from SFT training rows. The final pool (46,145 train, 720 monitor prompts) has **zero hash overlap** with SFT train, validation and test sources: English-source tasks use fresh English; m_hm and m_ood sources are the SFT model's own greedy translations of other fresh English sentences; hm_ood sources are its own hm outputs; 5,135 of 23,400 generated sources were removed by LID, length, loop, number, English-leak and format filters. The monitor set is fixed. An independent **reference monitor** of 300 SFT-validation prompts (60 per task, never trained on) reports chrF++ of the greedy final text, and a "best" checkpoint is saved only while it stays within 1.0 of the SFT model's 47.49.

6.2 Reward hacking with an evaluator-led reward (runs 1-2)

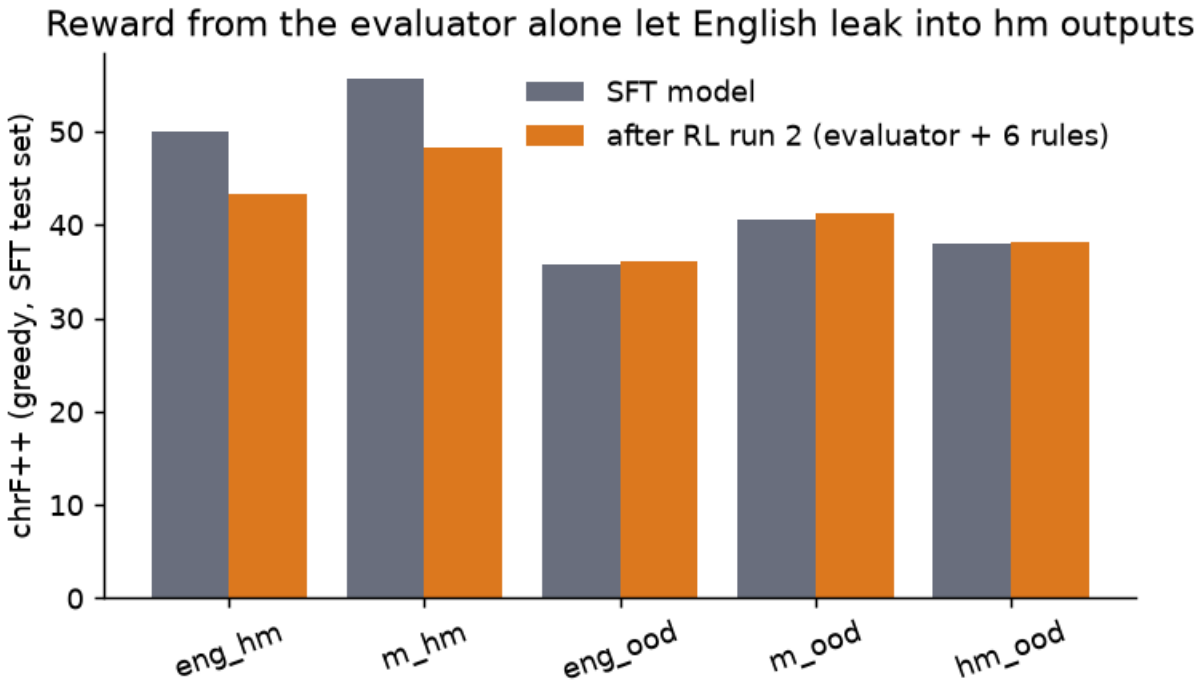


Fig. 4. chrF++ on the SFT test set before and after the evaluator-led RL run.

With the evaluator plus six rules (length, loop, number, English, switch share, draft) and no language-ID, function-word or diacritic checks, 540 steps raised the monitor reward from 0.37 to 0.61 (evaluator mean 3.14 to 3.48; format failures 9.6% to 0%). An independent check on the SFT test set told a different story (Fig. 4): chrF++ fell from 50.05 to 43.29 on eng_hm and from 55.58 to 48.35 on m_hm, with English function-word share in hm outputs up from about 0 to 0.08-0.10, while ood tasks were flat or slightly up (36.05 vs 35.69; 41.26 vs 40.49; 38.18 vs 37.96). The proxy improved and the gold-like metric degraded on exactly the tasks where the evaluator cannot see the failure. KL to the SFT model was about 0.15-0.17 throughout.

6.3 Rule-anchored RL (runs 6-7)

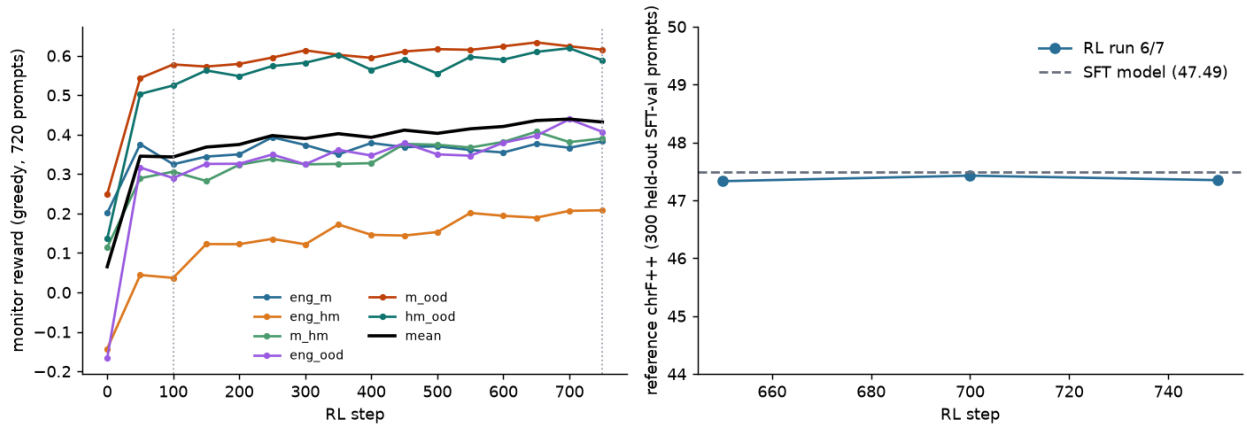


Fig. 5. Monitor reward by task (left) and held-out reference chrF++ (right). Dotted lines mark reward changes.

Training from the SFT model with the full reward and the fresh pool (about 1,500 logged steps over several resumed segments; the reward definition changed at step 100 and at step 750, so curves are not comparable across the dotted lines): the greedy monitor reward rose from 0.065 to about 0.43 and then plateaued (0.40-0.44 over the last 350 steps); format failures fell from 12% to 0.1%. KL to the SFT model stayed between 0.10 and 0.20 per token (Fig. 6) and the hard cut at 0.35 was never triggered. The 300-prompt held-out chrF

monitor is noisy (about +-1-1.5 chrF++ per task at 60 prompts per task), so checkpoints are compared on the full test set.

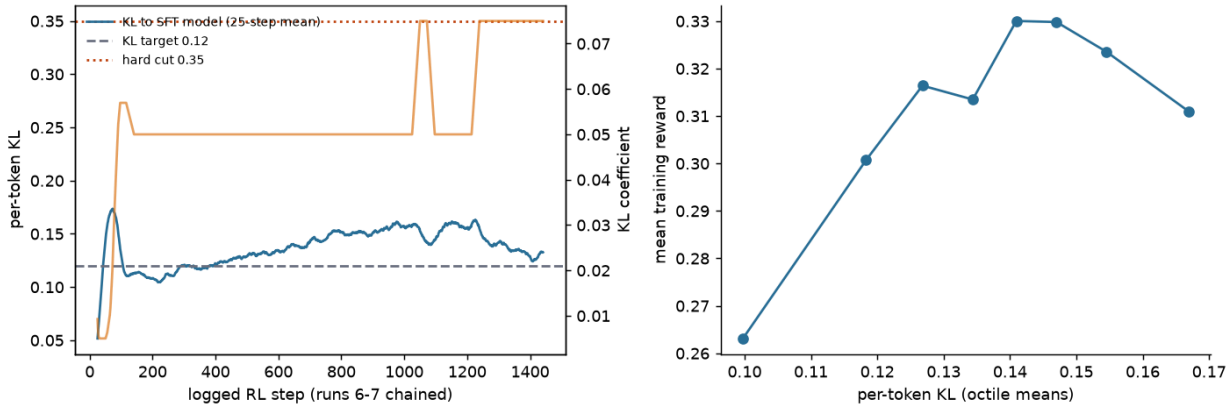


Fig. 6. Per-token KL to the SFT model and the adaptive KL coefficient across the chained runs (left); mean training reward by KL octile (right; confounded with training phase).

Full SFT test set (6,230 items, 34 languages; greedy decoding; chrF++ of the final text, macro over languages; the same vLLM decoder for all systems). "Best" is the highest-reward checkpoint that passed the per-task chrF guard; "latest" is the final policy.

task	SFT	RL best	RL latest	latest - SFT
eng_hm	50.05	48.78	49.38	-0.67
m_hm	55.58	54.02	54.57	-1.01
eng_ood	35.69	36.47	36.44	+0.75
m_ood	40.49	41.78	41.75	+1.26
hm_ood	37.96	38.77	38.82	+0.86

Format validity (one device block, two or more devices, non-empty text) rises from 0.949 and 0.954 (eng_hm, m_hm) to 0.996 and 0.999, and is 1.000 on the idiom tasks; the English-function-word excess in code-switching outputs stays at about zero (-0.011 and -0.003 against -0.003 and +0.003 for the SFT model), unlike run 2 (+0.08 and +0.105). The latest checkpoint is at least as good as "best" on every task (the guard, built on 60-prompt monitors, selected on noise).

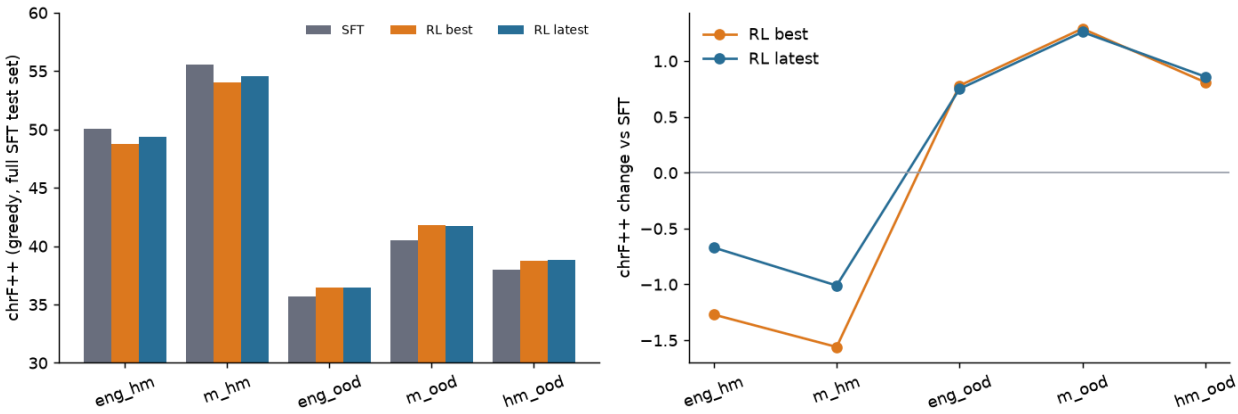


Fig. 7. chrF++ on the full SFT test set (left) and the change relative to the SFT model (right).

6.5 Where the reward gain comes from

We scored the same 6,230 test outputs of each system with the final evaluator and, with no weights applied, each rule (greedy decoding; no API calls).

task	evaluator (1-5): SFT -> RL latest	reward: SFT -> RL latest	format failures: SFT -> RL latest
eng_hm	3.013 -> 3.044	0.122 -> 0.283	2.0% -> 0.3%
m_hm	3.044 -> 3.099	0.141 -> 0.315	1.7% -> 0.1%
eng_ood	3.867 -> 4.173	0.473 -> 0.708	0.1% -> 0.0%
m_ood	3.891 -> 4.165	0.525 -> 0.742	0.1% -> 0.0%
hm_ood	3.715 -> 4.054	0.470 -> 0.704	0.0% -> 0.0%

Splitting the mean reward gain (+0.204 for latest, +0.210 for best; components sum to about 0.228 before clamping) gives: **evaluator score +0.054 (about 24-26%)**, **rule penalties +0.168 (about 74%)** and format gate +0.007 (about 3%). By task the evaluator accounts for 10-13% of the gain on the code-switching tasks and 32-36% on the idiom tasks. Most of the rule gain is device use (0.12 to 0.03 mean penalty), switch share (0.07 to 0.04), language ID (0.08 to 0.04), repetition (0.03 to 0.00), length and diacritics (Fig. 8). The model therefore did **not** improve "solely from the evaluator's feedback": the evaluator moved by +0.05 points on the 1-5 scale for code-switching and +0.3 for idioms, while most of the reward came from rules that the evaluator itself does not check. Whether the outputs are better in ways that matter is the question of the independent comparisons below.

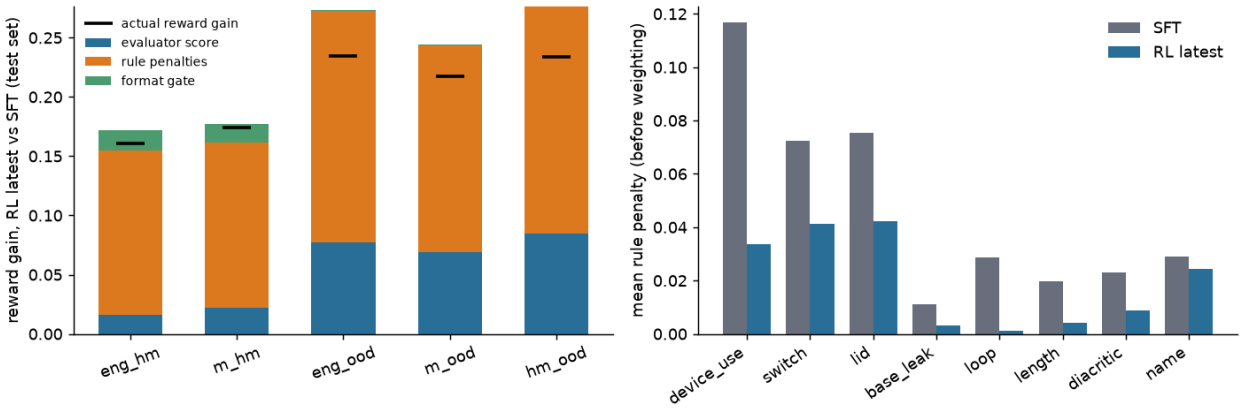


Fig. 8. Decomposition of the reward gain (left; the black bar is the actual gain, components are approximate because of clamping) and mean rule penalties before and after RL (right).

6.5b Decoding: greedy, sampling and best-of-8 with the evaluator as selector

For each test prompt we also drew eight samples (temperature 0.7, top-p 0.95) from the SFT model and from the latest RL policy, scored all nine candidates (greedy plus eight samples) with the final evaluator, and took the highest-scoring valid candidate (format failures rank last). No API calls were used. Corpus chrF++ is micro-averaged per task here, so absolute values are higher than the macro-over-languages numbers above; compare rows within the table.

system	task	greedy	random sample	evaluator pick	oracle (best chrF of 9)
SFT	eng_hm	55.14	52.47	52.78	58.49
SFT	m_hm	61.93	59.97	60.17	65.82
SFT	eng_ood	38.70	38.91	39.07	42.74
SFT	m_ood	40.89	40.77	40.94	44.67
SFT	hm_ood	39.65	39.89	39.97	43.59
RL latest	eng_hm	54.54	52.77	52.80	57.82
RL latest	m_hm	61.21	60.17	59.78	64.89
RL latest	eng_ood	38.60	37.89	37.82	41.57
RL latest	m_ood	41.37	40.48	40.48	44.09
RL latest	hm_ood	40.11	39.30	39.28	42.81

Findings: (i) sampling does not improve the per-sample chrF++: it lowers it on the code-switching tasks (by 2-3 points) and leaves it unchanged for the SFT model on the idiom tasks, and lowers it by about 0.8 for the RL policy, which was trained against greedy-heavy penalties; (ii) the evaluator as a selector raises the evaluator score of the pick (for example SFT eng_hm 3.01 to 3.13, eng_ood 3.87 to 4.26) and cuts format failures (SFT eng_hm 2.0% to 0.3%) but does not raise chrF++ above the greedy output, and for the RL policy it is slightly below greedy; (iii) there is substantial headroom (the oracle pick is 3-4 chrF++ above greedy), so a better selector, not more sampling, is what would help; the evaluator is not that selector. For human preference collection, sampled candidates add diversity without a systematic quality gain per sample, and the greedy output remains the strongest single candidate by this measure.

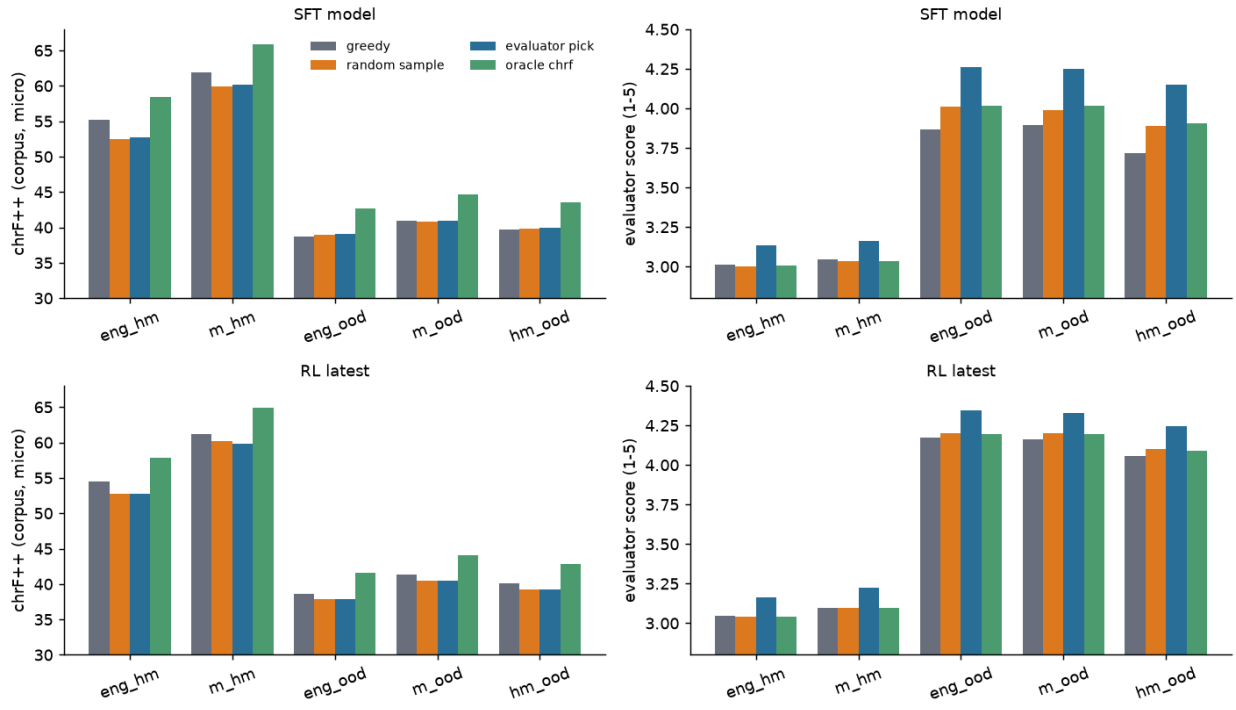


Fig. 9. Greedy, random sample, evaluator pick and oracle pick: chrF++ (left) and evaluator score (right) for the SFT model (top) and the latest RL policy (bottom).

6.5c Independent comparison with an LLM judge

To test whether the reward gains correspond to judged quality, we graded 1,000 held-out test items (200 per task, spread over languages, same seed for all systems) for four systems with Gemini 3.8 Flash on the Batch API: the SFT model, the best and the latest RL checkpoint (greedy outputs), and the data-generation model's

own curated text for the same items (the test references). The prompt is a standard, system-blind rubric (Appendix A): six criteria per task, integer anchors, no rule measurements, no forced grade distribution, requests of all systems shuffled together; 3,999 of 4,000 requests returned a parsable grade. Flash is also the judge that produced the evaluator's training labels, so this is not fully independent of the evaluator; cost limited us to one judge (actual batch cost $2.18).

task	SFT	RL best	RL latest	generator	RL latest - SFT (95% CI over items)
eng_hm	3.581	3.572	3.591	4.438	+0.010 [-0.073, +0.092]
m_hm	3.476	3.256	3.242	4.607	-0.235 [-0.338, -0.131]
eng_ood	2.315	2.310	2.338	3.672	+0.023 [-0.054, +0.097]
m_ood	2.385	2.458	2.434	3.695	+0.049 [-0.043, +0.144]
hm_ood	2.271	2.382	2.350	3.721	+0.079 [-0.006, +0.167]
all	2.805	2.795	2.791	4.027	-0.015 [-0.055, +0.025]

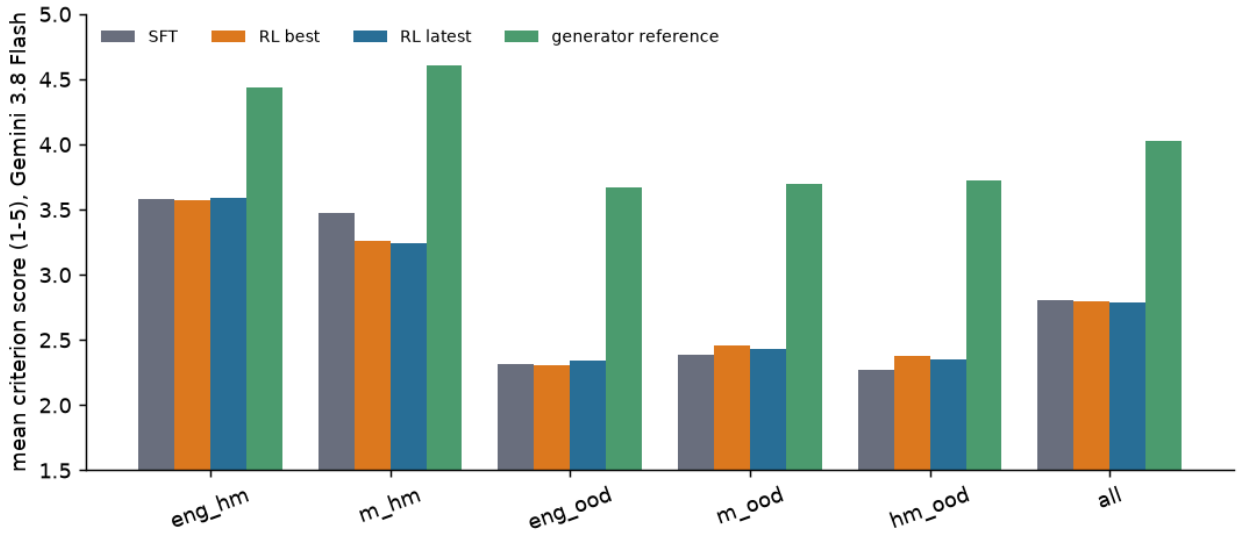


Fig. 10. Mean criterion score per system and task from Gemini 3.8 Flash with the standard prompt.

The reward gains do not appear as judged quality. Across all tasks the latest policy is indistinguishable from the SFT model (-0.015, CI includes zero); the only significant differences are a decline on m_hm (-0.24) and, for the best checkpoint, a small gain on hm_ood (+0.11 [+0.03, +0.20]). By criterion the code-switching outputs lose on meaning (3.24 to 3.04), switching naturalness (3.00 to 2.80) and overall (2.83 to 2.69) and are unchanged on language compliance (4.79); the idiom outputs gain on meaning (1.86 to 1.96), language correctness (2.34 to 2.43) and language compliance (4.65 to 4.81) but lose on idiom use (1.59 to 1.49). Head to head on the overall criterion, the latest policy beats SFT on 18-26% of items and loses on 17-33%, with 47-64% ties. Both models are far below the generator's own text (+1.2 points on average, 0.9-1.5 per task), which is the real quality gap. The rule penalties (format validity, repetition, device use, language ID) improved as intended, and the evaluator-selected gains are real in the sense of the reward, but they are not visible to an independent judge as better language.

Prompt sensitivity. We re-graded a 300-item subset with the old strict prompt used to produce the evaluator's labels (the same Flash model, 1,198 graded pairs). The two prompts agree at Spearman 0.84 across (item, system) pairs and rank the generator far above the models in both. The old prompt, which shows the judge our rule measurements, scores the RL outputs higher than SFT on the idiom tasks (+0.2 to +0.3 on the dimension mean) and eng_hm (+0.1), where the standard prompt sees no change: part of what the training labels, and therefore the evaluator and the reward, reward is the rule signal itself.

6.6 What the evaluator misses

On 86 logged eng_m samples in AfriCOMET languages the evaluator and AfriCOMET-QE agree with Spearman 0.53; the evaluator scores Nigerian Pidgin about 0.35 higher than QE (0.74 vs 0.39) because it does not penalise appended hallucinated sentences. This motivated the QE term and the per-language length windows. **[pending]** whether the QE term changes eng_m outputs measurably.

7. Discussion

What we claim.

RL with this reward improved what the reward measures (format validity, repetition, device use, language identity, evaluator score) but, in a single-judge comparison, left judged quality unchanged overall and lower on one code-switching task. A learned reward from a weak evaluator, plus a handful of rules, can be driven up while an independent reference metric falls, in a low-resource setting, and we found it only because an independent metric existed. Rules that encode what the evaluator cannot see (language identity, function words, repetition, names, devices) reduced it in our setting (run 2: -7 chrF++ on both hm tasks; runs 6-7 on the full test set: -0.7 and -1.0, with the idiom tasks improving by 0.8-1.3), but did not remove it. Most of the reward gain came from the rules, not from the evaluator; and the held-out reference monitor should be applied per task, not as an average, which hid the code-switching decline.

What we do not claim.

We did not test the hypothesis that overoptimisation sets in earlier or harder than with English reward models: there is no English control, one failure run, endpoint rather than curve measurements of the gold-like metric in run 2, and the independent higher-model gold judge for a proper proxy-versus-gold curve is paused. The monitor and the reward share the evaluator, so monitor gains are not independent evidence of quality; only held-out chrF++ is, and it measures similarity to SFT-style references, which RL is not trying to maximise.

8. Limitations and ethics

Small model; one seed per configuration; LLM-judge labels (no human ratings of code-switching quality), produced with a prompt that also showed the judge our own rule measurements (switch share, English-word share) and demanded that about half of all grades be 3 or lower, so the labels partly encode our rules and the evaluator cannot be read as an objective quality measure; cost limited thorough testing with several judges, so the independent comparison uses a single judge (Gemini 3.8 Flash), which also produced the evaluator's training labels and is therefore not fully independent of it; no human evaluation of code-switching quality (a blind candidate pool for human preference is released with the test set); the switch-share rule penalises 29-36% of human references; GlotLID is unreliable on short mixed spans; number-word and diacritic tables are incomplete (8 languages have no diacritic block, number words exist for about 30 languages and are sparse); the switch-language table is a human-supplied list that includes pairs a native speaker may find unnatural. Generated text may be wrong or offensive in languages we cannot fully review.

9. Availability

Weights and data are available to reviewers on request. This document and the accompanying results package (tables, figures, logs and model cards) contain everything needed to evaluate the claims made here.

Appendix A. Judge prompt (standard v2)

System prompt for the code-switching tasks (`{...}` are filled per item):

> You are an expert evaluator of written African-language text, with native-level knowledge of {language} and {switch_language}. A text-generation system was given an INPUT and asked to rewrite it as natural code-switched text that mixes {language} (the base language) with {switch_language}. You will see the INPUT, the meaning of the input in English, and the system's OUTPUT. Anchors (every criterion): 5 = fully correct, a native speaker would change nothing; 4 = minor issues that do not affect understanding; 3 = noticeable issues; 2 = serious issues that affect meaning or naturalness; 1 = unusable or wrong. Rate the OUTPUT only; ignore its length. Rate each criterion independently, using only what is written in the INPUT and OUTPUT. If you cannot judge a criterion in this language, give null; do not guess. Criteria: meaning (all information kept, nothing added or changed); base_language (grammar, word choice, spelling and orthography including diacritics of the {language} parts); switch_language (the same for the {switch_language} parts); language_compliance (every word is in {language} or {switch_language}; 1 = a third language or untranslated material dominates); switching_naturalness (switches happen where real bilingual speakers switch, in units that fit the grammar; 1 = random, forced or word-by-word substitution); overall. Reply with one JSON object: {"scores": {...}, "issues": [up to 3 short strings]}

For the idiom tasks the opening is "...rewrite it as informal, idiomatic {language}, using idioms, proverbs or expressions that native speakers really use", the criteria are meaning, language_correctness, idiom_use (real, apt and correctly used; null if it cannot be verified), naturalness, language_compliance (the base language, with occasional words of the languages allowed for it) and overall. The user message is a JSON object with the language, the switch language (code-switching tasks), the input as the system received it, the meaning of the input in English and the output (final text only; language tags and the device list removed). Temperature 0, thinking level low, one request per (item, system), all systems and tasks shuffled together.